WEEK 6 FINAL ASSIGNMENT

ARTIFICIAL INTELLIGENCE

Student Performance Prediction Using Machine Learning

Project Report

Project type: Supervised Machine Learning — Regression

Tools: Python, Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn, Jupyter Notebook

Prepared for: Week 6 AI Internship Final Project

Student name: Medisetti Jahnavi

Dataset transparency note: This report's numeric results and charts are generated from a deterministic synthetic dataset included with the notebook. The data is illustrative, not real student records. A documented UCI real-data extension is described in the report and notebook.

1. Introduction

Student performance can be influenced by several factors, including study habits, attendance, previous academic performance, assignment completion, sleep, access to learning resources, and support. A data-driven model can be used to explore patterns and estimate a final score from available features. This project demonstrates the end-to-end supervised learning workflow required for an introductory AI/ML project.

The project is framed as regression because the target is a numeric final score on a 0–100 scale. The prototype is intended for learning and academic support exploration; it should not be used to make high-stakes decisions about students.

2. Problem Statement and Objectives

Problem statement

Develop and evaluate a machine-learning model that estimates a student's final score using selected study, attendance, prior-score, assignment, sleep, and support indicators. The key technical challenge is preparing mixed numeric and categorical data and evaluating predictions on records held out from model training.

Objectives

- Load and inspect a structured dataset using Pandas and NumPy.
- Identify and handle missing values and duplicate records.
- Use exploratory data analysis to visualize score distribution and feature relationships.
- Build a reproducible preprocessing and Random Forest regression pipeline.
- Generate predictions and evaluate them with MAE, MSE, RMSE, and R^2.
- Document limitations, ethical considerations, and future improvements.

Why AI/ML is suitable

A supervised regression model learns relationships between input features and a known numeric outcome from labelled examples. Random Forest can represent non-linear patterns and interactions without requiring a linear relationship between every feature and the target. Its performance must still be compared against simple baselines and validated on representative data.

3. Dataset

The included file, **student_performance_dataset.csv**, contains 502 rows before cleaning and nine columns: eight predictors and one target. It is a deterministic synthetic dataset generated for reproducible practice. It intentionally includes a small number of missing values and duplicate rows so that preprocessing can be demonstrated. It does not represent actual students and must not be cited as real-world evidence.

Field	Meaning	Type
study_hours_per_day	Average daily study time	Numeric
attendance_percent	Attendance percentage	Numeric
previous_score	Prior academic score	Numeric
assignments_completed_out_of_10	Assignments completed, out of 10	Numeric
sleep_hours	Average sleep hours	Numeric
internet_access	Internet access available	Categorical
extracurricular_participation	Extracurricular participation	Categorical
parental_support	Reported support category	Categorical
final_score	Target score, 0–100	Numeric

Real-data option: the UCI Student Performance dataset is available from the UCI Machine Learning Repository at https://archive.ics.uci.edu/dataset/320/student+performance. It contains real educational records from a specific context; its scope, permissions, and feature availability should be checked before use. The included synthetic dataset and its metrics are kept separate from this optional dataset.

4. Data Preprocessing

The notebook first checks the dataset's shape, first and last records, data types, descriptive statistics, missing-value counts, and duplicate count. Duplicate rows are removed. Missing numeric values are filled with the median, while missing categorical values are filled with the mode for the initial cleaning demonstration.

Model-pipeline preprocessing

For model training, preprocessing is also implemented inside a Scikit-learn Pipeline and ColumnTransformer. Numeric columns use median imputation and standardization. Categorical columns use most-frequent imputation and one-hot encoding. Because these transformations are fitted within the training pipeline, test-set information is not used to learn imputation values or category mappings.

Dataset check: 502 rows were present before duplicate removal; 500 rows remained after duplicate removal. The raw file contains five deliberately introduced missing cells across three numeric and two categorical records. The notebook prints the actual before/after missing-value counts.

5. Exploratory Data Analysis

Four required visualizations are produced by the notebook. Their purpose is to understand the target distribution, compare groups, examine a possible relationship, and inspect numeric correlations before modelling.

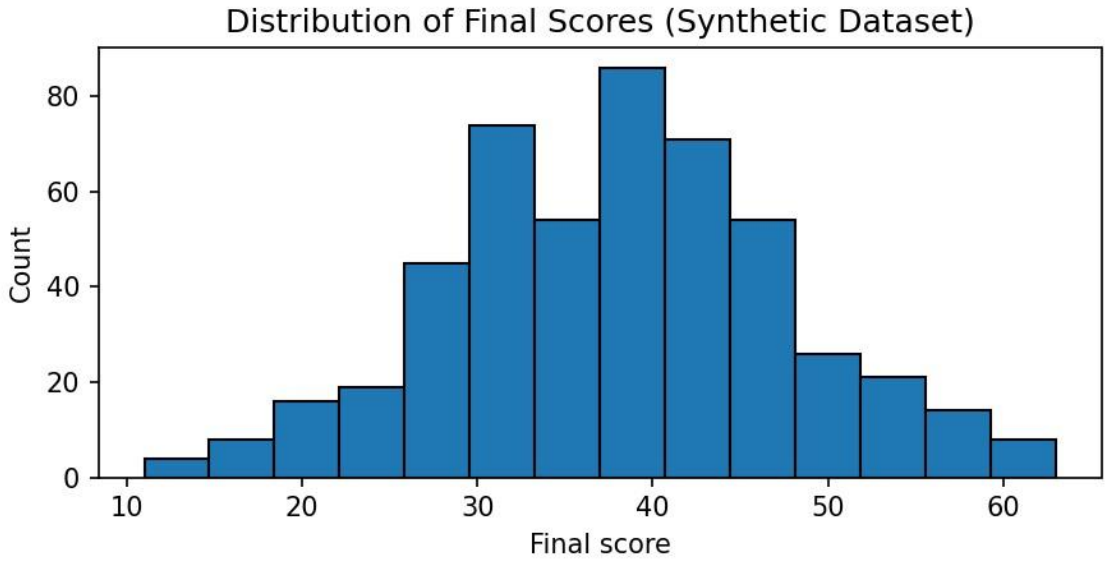


Figure 1. Distribution of final scores. The histogram shows the range and concentration of the synthetic target scores. It helps identify unusual values and whether the target distribution is skewed.

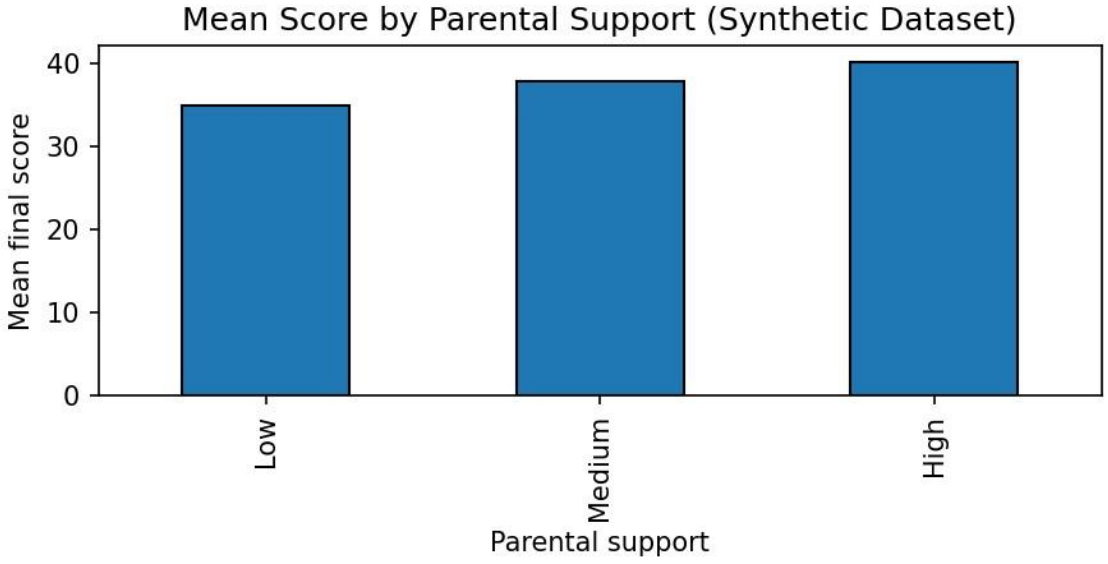


Figure 2. Mean score by parental-support category. The bars compare group means. The pattern reflects the synthetic data-generation design and does not establish causality or a real-world educational finding.

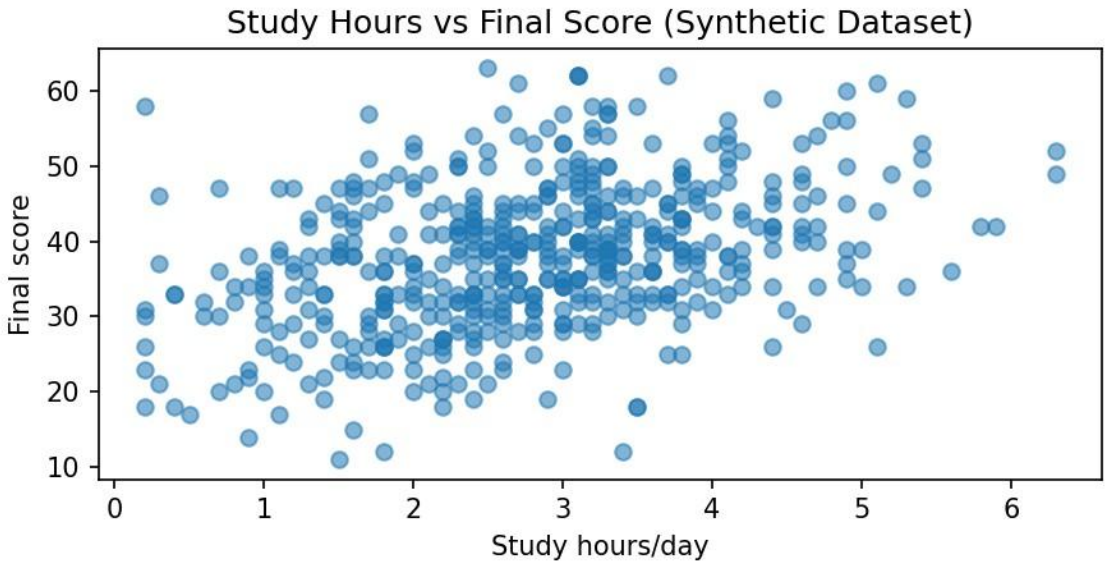


Figure 3. Study hours versus final score. The scatter plot shows the spread of scores at different study-hour values. It helps identify a trend while showing that study time alone does not determine the target.

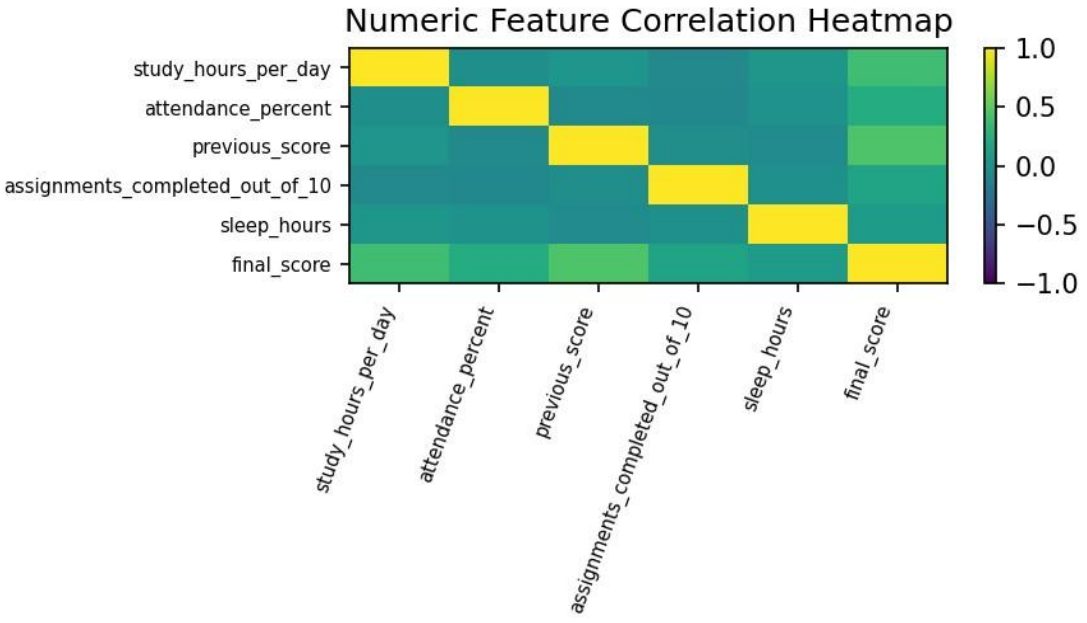


Figure 4. Correlation heatmap. Correlation coefficients summarize linear associations between numeric variables. Stronger correlation does not prove causation; values here are properties of synthetic data.

6. Model Development

The target variable is **final_score**, while the remaining eight columns are predictors. The data is split into 80% training and 20% testing records using random_state=42 so the split is reproducible. Random Forest Regressor is used with 250 trees and a minimum leaf size of two.

Why Random Forest?

- Captures non-linear relationships and interactions between features.
- Combines predictions from many decision trees, reducing reliance on a single tree.
- Can work with numeric and encoded categorical variables in one pipeline.
- Does not require normally distributed inputs, although it can still overfit or generalize poorly.

The model is trained only on the training partition. The test partition is reserved for evaluation after predictions are generated. A future improvement is to compare this model with a mean baseline and Linear Regression, and use cross-validation for more reliable estimates.

7. Training and Prediction

The notebook fits the preprocessing and estimator pipeline with **model.fit(X_train, y_train)** and generates test-set predictions with **model.predict(X_test)**. A table compares actual and predicted scores for sample test records.
Actual-versus-predicted and residual plots provide additional diagnostic checks.

With the current bundled dataset and fixed random seed, the cleaned dataset has 500 records; 400 records are used for training and 100 for testing. These counts and results correspond to the included synthetic sample only.

8. Model Evaluation

Regression metrics are used because the target is continuous. MAE and RMSE are expressed in score points; MSE is in squared score units; R^2 compares the model with a test-set mean baseline. Lower MAE/MSE/RMSE is better. R^2 closer to 1 indicates better fit relative to that baseline, while a negative R^2 indicates performance worse than the baseline.

Metric	Illustrative result	Interpretation
MAE	5.956	Average absolute error in score points
MSE	55.857	Average squared error
RMSE	7.474	Error in score points; penalizes larger misses
R^2	0.300	Relative fit versus predicting the test-set mean

Important: these are real computed outputs from the notebook's bundled synthetic sample, but they are not evidence of performance on real students. Do not claim that this model achieves the same scores on UCI or on institutional student data. When using another dataset, rerun the notebook and replace these results with that run's outputs.

Evaluation checks

- Inspect actual-versus-predicted scatter: points close to the diagonal indicate closer predictions.
- Inspect residuals: a systematic pattern can signal model misspecification; large residuals indicate larger errors.
- Compare against a simple baseline and use cross-validation before making performance claims.
- Check error patterns across relevant groups before any real deployment.

9. Conclusion

This project implements the full machine-learning workflow: problem definition, data inspection, cleaning, EDA, feature/target selection, train/test splitting, preprocessing, Random Forest training, prediction, and regression evaluation. The synthetic sample makes the notebook runnable offline and provides reproducible practice, but it cannot support claims about actual students. The principal achievement is a clear and reproducible end-to-end pipeline rather than a validated educational prediction system.

10. Future Scope

- Replace synthetic data with a documented, authorized real dataset and clearly define the intended prediction time.
- Compare Random Forest with Linear Regression, Decision Tree, and Gradient Boosting using cross-validation.
- Tune hyperparameters on training data only and preserve the test set for final evaluation.
- Report prediction intervals or uncertainty, not only point estimates.
- Review privacy, consent, fairness, subgroup error rates, and possible harms before any use with students.
- Build a simple interface for teachers to explore aggregate support needs, with human oversight and no automatichigh-stakes decisions.

References

- McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the Python in ScienceConference.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12,2825–2830.
- UCI Machine Learning Repository. Student Performance

  Dataset.https://archive.ics.uci.edu/dataset/320/student+performance • Pandas

  documentation: https://pandas.pydata.org/docs/

- Scikit-learn documentation: https://scikit-learn.org/stable/